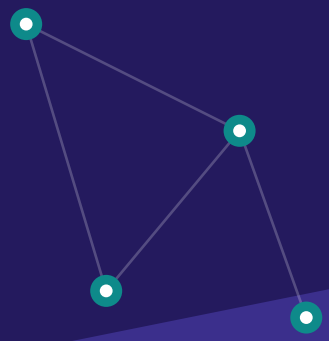
LANGGRAPH MASTERY SERIES



PHASE 4

Production-Grade Agents

Ship, observe, evaluate, and harden your agents.
Tracing, evals, deployment, and fault tolerance.

WHAT YOU WILL MASTER

- LangSmith tracing: full observability of every run
- Evaluation: datasets, evaluators, and LLM-as-judge
- LangSmith Deployment: your graph as a REST API
- Long-running & async execution, background runs
- Fault tolerance: retries, fallbacks, durable execution
- Auth & security: per-user tool scoping
- Versioning, cost, and latency optimisation

Four projects: trace a supervisor, run evals, deploy a REST API, ship a capstone.

Built on the official LangGraph documentation (docs.langchain.com) · LangGraph v1.1, 2026

From prototype to production

A graph that runs on your laptop is a prototype. A production agent is something else entirely: it is observed, tested, deployed behind an API, resilient to failure, scoped to the right user, and cheap enough to run at scale. This final phase closes the gap. It maps the official LangGraph and LangSmith tooling onto the seven concerns every team faces when an agent leaves the notebook — and ends with a capstone that ties the whole series together.

LangGraph gives you four production guarantees out of the box, and the rest of this phase shows how to use each one: **durable execution** (resume from the last checkpoint after a crash), **human-in-the-loop** (inspect and edit state mid-run), **comprehensive memory** (short-term and long-term), and **full observability** through LangSmith. The mindset shift is from “does it work once?” to “does it work reliably, measurably, and affordably, every time?”

> **The two halves of the stack**
>
> **LangGraph** is the open-source runtime — your graph, state, and nodes. **LangSmith** is the companion platform for tracing, evaluation, and deployment. They are separate products that integrate seamlessly: you build with LangGraph and operate with LangSmith.

1. LangSmith tracing — observability

A **trace** is the complete record of everything that happened during one run: every node, every LLM call, every tool invocation, with inputs, outputs, latency, and token counts. When an agent misbehaves in production, the trace is how you find out why. The remarkable part is how little code it takes: because LangGraph is built on LangChain runnables, tracing is enabled with two environment variables and zero code changes.

Enable tracing. No code change — just run your graph as normal.

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>

# Optional: name the project traces land in (defaults to "default")
export LANGSMITH_PROJECT=my-agent-project
```

With those set, every `invoke` / `stream` call on a compiled graph is logged to LangSmith automatically, with the full node-by-node tree visible in the UI. To trace only some calls, wrap them in a context manager instead of relying on the global variable.

Selective tracing plus tags and metadata for filtering.

```
import langsmith as ls

# This run WILL be traced, regardless of the env var:
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Summarise Q3"}]})

# Attach tags and metadata to make traces searchable in the UI:
agent.invoke(
    {"messages": [{"role": "user", "content": "Summarise Q3"}]},
```

```
    config={
        "tags": ["production", "report-agent", "v1.0"],
        "metadata": {"user_id": "user_123", "environment": "production"},
    },
)
```

> **Tag everything you will want to filter by**
>
> User id, environment, model version, feature flag. In production you will have thousands of traces; tags and metadata are how you find the ten that went wrong. Set `LANGSMITH_PROJECT` per environment so staging and prod never mix.

Sensitive data is a real concern once traces leave your process. LangSmith supports **anonymizers** — rule-based redaction applied before anything is sent — so you can mask PII such as social-security numbers, emails, or API keys.

Redact SSNs (and anything else) before traces are stored.

```
from langchain_core.tracers.langchain import LangChainTracer
from langsmith import Client
from langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([
    {"pattern": r"\b\d{3}-?\d{2}-?\d{4}\b", "replace": "<ssn>"},
])
tracer = LangChainTracer(client=Client(anonymizer=anonymizer))

graph = builder.compile().with_config({"callbacks": [tracer]})
```

2. Evaluation — measuring reliability

LLMs are non-deterministic: a small change to a prompt, model, or tool can quietly break behaviour you thought was solid. **Evaluation** is how you catch that. The official workflow has exactly three pieces, and once you internalise them everything else is detail.

Component	What it is
Dataset	A set of test inputs, optionally with expected (reference) outputs.
Target function	The thing under test — one prompt, one node, or the whole agent.
Evaluators	Functions that score the target's output against the reference.

You build a dataset once with the LangSmith client, then run it against your agent as often as you like — on every prompt change, every model swap, every release.

Create a labelled dataset once; reuse it forever.

```
from langsmith import Client
ls_client = Client()

examples = [
    {"inputs": {"text": "Shut up, idiot"},        "outputs": {"label": "Toxic"}},
    {"inputs": {"text": "You're a wonderful person"}, "outputs": {"label": "Not toxic"}},
    {"inputs": {"text": "Nobody likes you"},      "outputs": {"label": "Toxic"}},
]
```

```
dataset = ls_client.create_dataset(dataset_name="Toxic Queries")
ls_client.create_examples(dataset_id=dataset.id, examples=examples)
```

An **evaluator** is just a function. It receives the example inputs, your agent's actual outputs, and (when present) the reference outputs, and returns a score. The simplest is exact match; the most flexible is an LLM-as-judge for open-ended text that has no single right answer.

Score the agent across the dataset with evaluate().

```
# A simple deterministic evaluator: does the class match the label?
def correct(inputs: dict, outputs: dict, reference_outputs: dict) -> bool:
    return outputs["class"] == reference_outputs["label"]

# Run the evaluation — creates an "experiment" you can inspect in the UI:
results = ls_client.evaluate(
    my_agent,                       # target: takes inputs, returns outputs
    data=dataset.name,
    evaluators=[correct],
    experiment_prefix="baseline",   # names the experiment
    max_concurrency=4,              # parallelise across the dataset
)
```

LLM-as-judge and trajectory evals

For conversational output with no exact answer, an **LLM-as-judge** evaluator grades against a rubric. For agents specifically, **trajectory** evaluators score the sequence of tool calls — did the agent take the right steps, not just reach a plausible answer? The open-source `openevals` and `agentevals` packages ship ready-made versions of both.

Use aevaluate() for large jobs

`evaluate()` and its async twin `aevaluate()` have identical interfaces. For anything beyond a few dozen examples, reach for the async version and tune `max_concurrency` to keep wall-clock time down.

3. LangSmith Deployment — your graph as an API

Deployment turns a compiled graph into a managed, horizontally-scaled service with a REST API, persistence, streaming, and a task queue for long-running runs — without you writing a server. It is the same runtime that powers LangGraph Studio locally, scaled for production.

Everything hinges on one configuration file, `langgraph.json`, which tells the platform where your graph lives, what it depends on, and where to read environment variables. You wrote this file in Phase 3 to use Studio; the very same file deploys to the cloud.

langgraph.json — the single source of truth for both Studio and deploy.

```
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./my_agent.py:graph"
  },
  "env": ".env"
}
```

Run it locally first with the dev server, then connect from any client using the SDK. The SDK speaks to the same REST endpoints whether you point it at localhost or a cloud deployment, so code you write against the dev server works unchanged in production.

Drive a deployed graph through the REST API with the official SDK.

```
# Local dev server (hot-reloads your graph, opens Studio):
#   langgraph dev

from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

# Stream a run against the deployed "agent" graph over REST:
for chunk in client.runs.stream(
    None,                       # threadless run (stateless)
    "agent",                    # the graph name from langgraph.json
    input={"messages": [{"role": "user", "content": "Hello!"}]},
    stream_mode="messages",
):
    print(chunk.event, chunk.data)
```

> **From local to cloud**
>
> To deploy for real, push your project (with `langgraph.json`) to GitHub and create a deployment from the LangSmith UI pointing at the repo. You get a hosted URL, the same `/runs/stream` endpoints, plus persistence and scaling managed for you. Swap the `url` above and nothing else changes.

4. Long-running tasks & async execution

Real agents wait — on slow tools, on human approval, on multi-minute research. Blocking a web request for that long is untenable, so LangGraph leans on two capabilities. First, every graph has an async API: `ainvoke` and `astream` mirror their sync counterparts and let one process handle many concurrent runs.

The async API: ainvoke / astream for concurrent, non-blocking runs.

```
# Async invocation — non-blocking, ideal for servers handling many users:
result = await graph.ainvoke(
    {"messages": [{"role": "user", "content": "Research and summarise"}]},
    config={"configurable": {"thread_id": "user-42"}},
)

# Async streaming of incremental updates:
async for chunk in graph.astream(inputs, config, stream_mode="updates"):
    print(chunk)
```

Second, when work genuinely outlasts a request, the deployment platform runs it as a **background run**: you kick off the run, get an id back immediately, and poll or stream for results later. Combined with checkpointers, this is what lets an agent pause for hours at a human-approval interrupt and resume exactly where it left off — the **durable execution** guarantee in action.

> **The thread is the unit of continuity**
>
> A `thread_id` plus a checkpointer means any run can be paused, survive a process restart, and resume from the last completed super-step. Long-running and fault-tolerant are two views of the same persistence machinery you met in Phase 2.

5. Fault tolerance — retries, fallbacks, recovery

By default, when a node raises, the graph stops and the error surfaces immediately. This is deliberate: LangGraph never silently retries or swallows failures. You opt into resilience explicitly, and there are three complementary tools for doing so.

Retries — for transient failures

Attach a `RetryPolicy` to any node. Network blips and rate limits resolve themselves on a second attempt; the policy controls how many attempts, how long to wait, and which exceptions qualify.

RetryPolicy makes reliability a graph-level setting, not node clutter.

```
from langgraph.types import RetryPolicy

builder.add_node(
    "search_documentation",
    search_documentation,
    retry_policy=RetryPolicy(
        max_attempts=3,        # including the first try
        initial_interval=1.0,  # seconds before first retry
        backoff_factor=2.0,    # exponential back-off
        retry_on=ConnectionError,
    ),
)
```

Fallbacks — for when a model or tool is down

LangChain runnables support `.with_fallbacks()`: if the primary fails, the next in the chain takes over. This is the clean way to fail over from a premium model to a cheaper or more available one.

Automatic model fail-over with .with_fallbacks().

```
primary  = init_chat_model("anthropic:claude-sonnet-4-6")
backup   = init_chat_model("anthropic:claude-haiku-4-5")
resilient_model = primary.with_fallbacks([backup])
```

Recovery loops — for errors the LLM can fix

Some failures are best handed back to the model. Catch the error, write it into state, and route back so the agent can read what went wrong and try a different approach — the `Command` pattern from Phase 3 doing double duty as an error handler.

Loop errors back to the agent so it can self-correct.

```
from langgraph.types import Command

def execute_tool(state: State) -> Command:
    try:
        result = run_tool(state["tool_call"])
        return Command(update={"tool_result": result}, goto="agent")
    except ToolError as e:
```

```
        # Let the LLM see the failure and decide what to do next:
        return Command(update={"tool_result": f"Tool error: {e}"}, goto="agent")
```

Pending writes: parallel failures are cheap to retry

If one branch of a parallel super-step fails, LangGraph keeps the successful siblings' outputs as **pending writes** and does not re-run them on retry. You only pay to redo the branch that actually failed — durable execution at the super-step level.

Code before interrupt()/retry must be idempotent

On resume or retry, a node restarts from the top of the function. Any side effect that runs before the pause — charging a card, sending an email — will run again. Make such effects idempotent, or place them after the interrupt.

6. Auth & security — scoping tools per user

In multi-tenant production, the agent must act *as* the requesting user, never beyond their permissions. Two mechanisms combine to enforce this. First, runtime **context** carries the authenticated identity into every node — never trust an id the model wrote into the conversation.

Carry trusted identity via typed runtime context, not the prompt.

```
from dataclasses import dataclass
from langgraph.runtime import Runtime

@dataclass
class Context:
    user_id: str
    scopes: list[str]      # what this user is allowed to do

def agent_node(state: State, runtime: Runtime[Context]):
    user = runtime.context.user_id        # trusted, injected at invoke time
    scopes = runtime.context.scopes
    ...

graph.invoke(
    {"messages": [...]},
    context=Context(user_id="user_123", scopes=["read:reports"]),
)
```

Second, use that context to **scope tools dynamically**: filter the toolset, or refuse a call, based on the user's permissions before the model ever sees or runs a tool. A user with read-only scope simply never gets the delete tool bound.

Per-user tool scoping: bind only the tools the caller is allowed.

```
def tools_for(scopes: list[str]) -> list:
    tools = [search_reports]                    # everyone can read
    if "write:reports" in scopes:
        tools.append(edit_report)               # only writers can edit
    if "admin" in scopes:
        tools.append(delete_report)             # admins only
    return tools

def agent_node(state, runtime: Runtime[Context]):
    model = base_model.bind_tools(tools_for(runtime.context.scopes))
```

```
    return {"messages": [model.invoke(state["messages"])]}
```

> **Defence in depth**
> Scope tools at bind time AND re-check inside the tool itself. The model can hallucinate a tool call; the deployment platform adds its own auth layer on the API. Assume each layer can fail and verify permissions where the action actually happens.

7. Versioning, cost & latency

Two production concerns remain, and the same tooling addresses both. **Versioning**: because behaviour can shift with any prompt or model change, treat your agent like code. Tag every deployment, stamp runs with a version in trace metadata, and gate releases behind the evaluation suite from section 2 — an experiment per version, compared side by side in LangSmith.

Cost and latency are levers you tune deliberately, measured against the traces and evals you now collect. The highest-impact moves, roughly in order:

- **Cache** deterministic or repeated work with a node `CachePolicy` so identical inputs skip the model entirely.
- **Right-size the model** per node — a small fast model for routing and classification, a frontier model only where reasoning genuinely needs it.
- **Parallelise** independent work with the map-reduce / Send pattern from Phase 3: latency becomes the slowest branch, not the sum of all branches.
- **Stream** so users see progress immediately — perceived latency falls even when total time is unchanged.
- **Bound the loop** with a recursion limit and explicit iteration caps so a confused agent can never run up an unbounded bill.

> **Measure, then optimise**
> Every lever above is only as good as your visibility into it. Traces give you per-node latency and token counts; evals tell you whether a cheaper model actually held quality. Optimise against data, never against a hunch — that is the whole point of phases 1 and 2 of this chapter.

Projects: ship it for real

Four projects take you from an observed agent to a fully deployed, evaluated capstone. Each builds on the Phase 3 supervisor, so keep that code close.

PROJECT 1

Add full LangSmith tracing to your supervisor

Goal: Make every run of the Phase 3 supervisor observable, tagged, and searchable.

No graph changes are needed — only configuration. Set the environment, then enrich each run with tags and metadata so you can later filter by user, version, or environment.

Tracing is configuration, not code: tag runs for later analysis.

```
import os
import langsmith as ls
from phase3_supervisor import supervisor   # your compiled Phase 3 graph

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_API_KEY"] = "<your-key>"
os.environ["LANGSMITH_PROJECT"] = "supervisor-prod"

def run(question: str, user_id: str, version: str = "v1.0"):
    return supervisor.invoke(
        {"messages": [{"role": "user", "content": question}]},
        config={
            "tags": ["supervisor", version],
            "metadata": {"user_id": user_id, "environment": "production"},
        },
    )

if __name__ == "__main__":
    out = run("Write a short brief on LangGraph", user_id="user_7")
    print(out["messages"][-1].content)
```

Open LangSmith and inspect the trace tree: the supervisor node, each worker subgraph, every LLM call and tool result, with latency and token counts at every level. This single view is the fastest way to understand — and debug — a multi-agent system.

PROJECT 2

Write an evaluation dataset and run evals

Goal: Quantify whether your supervisor actually produces good answers — and keep it that way.

Build a small labelled dataset of questions with reference answers, define an evaluator, and run it against the supervisor. Start with a deterministic check, then graduate to an LLM-as-judge for the open-ended writing quality.

A dataset, a target, an evaluator — the three pieces of every eval.

```
from langsmith import Client
from phase3_supervisor import supervisor

ls_client = Client()

examples = [
    {"inputs": {"q": "What is a LangGraph subgraph?"},
     "outputs": {"must_mention": "compiled graph used as a node"}},
    {"inputs": {"q": "When use a supervisor?"},
     "outputs": {"must_mention": "delegates to specialised workers"}},
]
dataset = ls_client.create_dataset(dataset_name="Supervisor QA")
ls_client.create_examples(dataset_id=dataset.id, examples=examples)

def target(inputs: dict) -> dict:
    out = supervisor.invoke(
        {"messages": [{"role": "user", "content": inputs["q"]}]})
    return {"answer": out["messages"][-1].content}

def mentions_key_idea(inputs, outputs, reference_outputs) -> bool:
    needle = reference_outputs["must_mention"].lower()
    return needle in outputs["answer"].lower()

results = ls_client.evaluate(
    target,
    data=dataset.name,
    evaluators=[mentions_key_idea],
    experiment_prefix="supervisor-v1",
    max_concurrency=4,
)
print("Done — view scores in LangSmith.")
```

Re-run on every change

Change a prompt? Swap a model? Re-run this exact script. Two experiments appear side by side in LangSmith and you can see immediately whether the change helped or quietly regressed.

PROJECT 3

Deploy your agent via the LangGraph Platform

Goal: Expose the supervisor as a REST API and drive it from a client over HTTP.

Add a `langgraph.json`, run the dev server, then call it through the SDK exactly as a production client would. The same code works against a cloud deployment later.

Point the platform at your compiled supervisor graph.

```
// langgraph.json  (project root)
{
  "dependencies": ["."],
  "graphs": { "agent": "./phase3_supervisor.py:supervisor" },
  "env": ".env"
}
```

Run the dev server, then stream from the REST API via the SDK.

```
# Terminal 1 — start the server (also opens Studio):
#   langgraph dev

# client.py — Terminal 2:
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",                                  # graph name from langgraph.json
    input={"messages": [{"role": "user",
                         "content": "Brief me on multi-agent systems"}]},
    stream_mode="messages",
):
    print(chunk.event, chunk.data)
```

To go live, push the project to GitHub and create a deployment from the LangSmith UI. You receive a hosted URL with the same endpoints, plus managed persistence and scaling — change the `url` and your client is unchanged.

PROJECT 4

Capstone — a complete end-to-end agentic system

Goal: Combine memory, tools, human approval, and observability into one production agent.

This is the synthesis of the entire series. Build a single agent that exhibits every production property you have learned, then deploy and evaluate it. The skeleton below wires the pieces together; fleshing out the tools and prompts is the exercise.

The capstone: memory, tools, human approval, retries, and tracing in one graph.

```
import os
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import interrupt, Command, RetryPolicy
from langchain.chat_models import init_chat_model

os.environ["LANGSMITH_TRACING"] = "true"        # (4) observability

model = init_chat_model("anthropic:claude-sonnet-4-6")

def send_email(to: str, body: str) -> str:
    """Send an email after human approval."""
    approved = interrupt({"action": "send_email", "to": to, "body": body})
    if not approved:                            # (3) human-in-the-loop
        return "Cancelled by reviewer."
    return f"Sent to {to}."

tools = [send_email]                            # (2) tools
llm = model.bind_tools(tools)

def agent(state: MessagesState):
    return {"messages": [llm.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("agent", agent,
                 retry_policy=RetryPolicy(max_attempts=3))   # (5) resilience
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", tools_condition)
builder.add_edge("tools", "agent")

with SqliteSaver.from_conn_string("capstone.db") as saver:   # (1) memory
    app = builder.compile(checkpointer=saver)
    cfg = {"configurable": {"thread_id": "user-1"},
           "tags": ["capstone"], "metadata": {"user_id": "user-1"}}
    # Run until the approval interrupt, then resume with a decision:
    app.invoke({"messages": [{"role": "user",
                "content": "Email alice@x.com a thank-you note"}]}, cfg)
    app.invoke(Command(resume=True), cfg)       # approve and continue
```

Every numbered concern is present: persistent memory via the SQLite checkpointer and a `thread_id`; tools the model can call; a human-approval interrupt inside the sensitive tool; retry-based fault tolerance; and full tracing through the environment variable. Add the `langgraph.json` from Project 3 and you can deploy this same graph as an API; add the dataset from Project 2 and you can evaluate it on every change.

> **You have completed the series**
> From the agent loop in Phase 1, through LangGraph's primitives in Phase 2 and composition patterns in Phase 3, to production hardening here — you now have a complete mental model and a working toolkit for building, shipping, and operating reliable LLM agents with LangGraph.

Where to go next: deepen any single pillar against the official docs at `docs.langchain.com` — durable execution, long-term memory stores, online (production) evaluation, and the deployment platform each have rich guides. The patterns in this series are the foundation; the docs are the reference you will return to as your agents grow.